

目 录

[概述](#)

[入门指南](#)

[Demo示例](#)

概述

纳博特是国内领先的工业机器人控制器厂商。纳博特自主研发的开放型 NexDroid 软件平台，可以提供稳定可靠，功能丰富的二次开发接口(NexDroidAPI)。NexDroid 分离了运控算法层与应用层，让合作伙伴只需关注行业需求。利用 NexDroidAPI，结合行业经验，就可开发专有自动化装备，如3C电子、医疗、核能、工程机械、风电等行业设备。

纳博特文档中心提供了基于纳博特控制系统的示例Demo、二次开发接口说明以及二次开发资源下载，使开发者可以全面了解纳博特控制系统的能力，降低开发门槛，提高开发效率，使开发者能够基于纳博特控制系统迅速构建出针对场景终端应用的机器人产品。

INEXBOT
纳 博 特 科 技



纳博特软件平台提供如下层级的二次开发，用户可按需选择。

- 控制器：

专有工艺开发，特殊机型支持，插补算法替换等。Linux系统，C++开发。

- 示教器：

定制专属UI/UE。配合控制器二次开发，实现专有功能定制。Linux系统，Qt/C++开发。

- 上位机：

抛开示教器束缚，通过api直接硬控机器人。Windows系统：VC++/C#/VB/Python/Qt；Linux系统，C++/Qt/Python

- 主 站：

纳博特控制器退化为工控机，提供实时系统和Ethercat主站功能，满足客户随心定制。Linux系统，C/C++开发。

- JSON协议：

遵循协议就可以愉快操控纳博特控制器。Windows系统：VC++/C#/VB/Python/Qt；Linux系统，C++/Qt/Python

- ROS:

纳博特控制器按照ROS协议实现为一个Node，你可以按ROS协议来控制它。

目标受众

- 本体厂商：

客户可基于纳博特系统，快速客制化其控制系统，打造自有机器人大脑，提高品牌辨识度。

- 集成商：

开发专有工艺，保护自有行业知识，打造产品护城河。

- 科研用户：

基础功能交给纳博特，客户专注高精运控算法研究，深挖机器人极限性能，勇攀科技高峰。

- 教育用户：

让学生不仅会用机器人，更会造机器人。

主要内容

- Demo示例

纳博特Demo示例提供了提供多个基于纳博特控制器典型应用的Demo，为开发者利用纳博特控制系统进行应用开发提供参考。纳博特Demo示例提供了支持C、C++、Qt、C#、Python、ROS等环境的Demo示例，用户可以在常见的开发环境中进行快速演示。纳博特Demo示例内容覆盖机械臂的各种基础操作和高级功能，包括坐标系操作、力控、抓取、IO功能、外部轴控制、ModbusRTU通信、样条曲线运动、角度透传、在线编程以及算法等，使用户可以快速掌握各种API接口的应用。

- 二次开发接口

纳博特机械臂二次开发接口将机械臂一些复杂的功能进行接口化，用户无需进行底层的开发即可快速使用机械臂的高级控制功能，降低了用户基于纳博特控制器的开发门槛，开发者能够轻松实现应用程序开发。纳博特控制器二次开发接口支持C、C++、Qt、C#、LabVIEW、Python、JSON、ROS等开发环境，充分满足用户任意环境的部署。纳博特二次开发接口涵盖详细的接口说明、参数解释和示例代码，用户可以全面了解每个接口的功能和用法。

- 相关资源获取

软件二次开发资源包提供了API接口包、SDK包、ROS包等下载资源，基于软件包用户可以便捷地进行二次开发，满足用户快速进行应用部署。

联系方式和支持

- 了解更多纳博特产品，请访问：[纳博特运动控制官网](#)
- 了解更多纳博特产品的使用，请访问：[纳博特运动控制-哔哩哔哩](#)
- 咨询纳博特产品相关问题，请发送邮件至官方邮箱: sales@inexbot.com
- 纳博特产品的最新资讯，获取产品详细说明书、手册：官方微信公众号、官方微信小程序





入门指南

不同二次开发形式入门指南

纳博特软件平台提供多种二次开发形式，旨在满足用户对控制器、示教器、上位机和主站的不同需求。根据不同的应用场景，用户可以选择适合的开发方式进行功能定制。本指南将帮助用户根据需求选择合适的开发形式，并简要说明相关的开发知识要求。

1. 控制器二次开发

应用场景：专有工艺开发、特殊机型支持、插补算法替换等。

开发方式：

- **操作系统**：Linux 系统
- **编程语言**：C++
- **主要功能**：开发控制器底层算法和特定硬件的支持，如替换插补算法、定制硬件接口等。
- **开发内容**：该开发方式要求用户有较强的底层硬件和算法理解。通常适用于机器人控制器的核心功能定制，例如，调整路径规划、实时控制优化等。

必须的知识：

- **C++ 编程**：需要熟悉 C++ 语言，掌握面向对象编程和数据结构。
- **Linux 系统编程**：对 Linux 系统的基本操作（如进程管理、内存管理等）有一定了解。
- **机器人控制理论**：了解机器人控制算法（如运动学、动力学、插补算法等）有助于理解控制器开发中的关键问题。

适合的用户：控制器硬件和算法开发人员，具有较强的编程能力，能够与硬件交互并优化机器人控制流程。

2. 示教器二次开发

应用场景：定制专属用户界面（UI/UE）和功能，实现与控制器的协同工作。

开发方式：

- **操作系统**：Linux 系统
- **编程语言**：Qt / C++
- **主要功能**：定制化的用户界面设计，结合控制器的二次开发，实现专有功能。例如，设计控制面板、手动操作界面，配合控制器定制动作和命令输入。
- **开发内容**：需要通过 Qt 框架实现图形界面的开发，并与控制器的功能进行结合。

必须的知识：

- **Qt 框架**：熟悉 Qt 的 UI 设计与开发，能够使用 Qt Designer 和 QML 等工具进行界面设计。
- **C++ 编程**：掌握 C++ 语言，理解 Qt 的信号与槽机制、事件处理等。
- **Linux 系统开发**：基础的 Linux 系统操作，特别是对于与控制器进行交互的部分。

适合的用户：UI/UE 开发人员，具备前端开发经验，想要为用户提供自定义操作界面的工程师。

3. 上位机开发

应用场景：实现直接通过 API 控制机器人，摆脱示教器限制，进行更灵活的控制。

开发方式：

- **操作系统：**
 - **Windows 系统：**VC++ / C# / VB / Python / Qt
 - **Linux 系统：**C++ / Qt / Python
- **主要功能：**通过 API 直接控制机器人，可以在上位机上进行更高级的控制和监控。支持编程语言多样性，可以根据项目需求灵活选择开发语言和工具。
- **开发内容：**通过网络接口或串口等通信方式与控制器进行交互，发送指令并获取反馈信息。此开发方式不依赖于示教器，适合对机器人进行复杂的远程控制和数据处理。

必须的知识：

- **编程语言选择：**需要根据操作系统和语言选择进行学习。对于 Windows 系统，用户需要了解 C#、VC++ 或 Python 等语言；对于 Linux 系统，用户需要熟悉 C++、Qt 或 Python。
- **API 调用：**了解如何通过 API 与机器人控制器进行交互。
- **网络编程：**掌握基础的网络编程知识，尤其是在分布式系统中如何进行设备间的通信。

适合的用户：需要进行机器人高层控制、自动化操作或系统集成的开发人员。

4. 主站开发

应用场景：将控制器退化为工控机，提供实时系统和 Ethercat 主站功能，实现客户需求的定制化。

开发方式：

- **操作系统：**Linux 系统
- **编程语言：**C / C++
- **主要功能：**通过主站功能实现 Ethercat 主站与设备的通信，满足客户对机器人控制的实时性需求。可以在该平台上开发实时控制系统和工业网络通信协议。
- **开发内容：**编写实时任务、开发 Ethercat 主站协议，进行与控制器和其他设备的通信。

必须的知识：

- **C/C++ 编程：**深刻理解 C/C++ 编程语言，尤其是实时编程和并发控制。
- **实时操作系统 (RTOS)：**掌握实时系统开发，理解实时任务调度、系统性能等方面的知识。
- **Ethercat 协议：**了解 Ethercat 协议和工业网络通信，能够实现设备间的实时高效通信。

适合的用户：工业自动化和嵌入式系统开发人员，尤其是需要定制实时控制系统和网络通信协议的工程师。

5. JSON 协议开发

应用场景：通过遵循 JSON 协议与纳博特控制器进行通信，进行远程控制和数据交换。

开发方式：

- **操作系统：**
 - **Windows 系统：**VC++ / C# / VB / Python / Qt
 - **Linux 系统：**C++ / Qt / Python
- **主要功能：**基于 JSON 协议，开发系统与机器人之间的通信接口，可以方便地进行远程控制和数据交互。
- **开发内容：**通过 JSON 格式的消息与机器人控制器进行交互，发送控制指令和获取执行结果。

必须的知识：

- **JSON 格式：**理解 JSON 数据结构，能够编写与控制器通信的 JSON 消息。
- **编程语言：**根据系统平台选择适合的编程语言进行开发。
- **网络通信：**掌握基于 HTTP、TCP/IP 等协议的网络通信方式。

适合的用户：需要通过标准化协议与机器人进行集成或远程控制的开发人员。

6. ROS 开发

应用场景：基于 ROS 协议开发和控制纳博特控制器，进行更灵活的机器人控制和调试。

开发方式：

- **操作系统：**Linux 系统
- **编程语言：**C++ / Python
- **主要功能：**通过 ROS（机器人操作系统）控制纳博特控制器，能够集成到 ROS 网络中，支持机器人系统的实时监控和控制。
- **开发内容：**开发 ROS 节点，与其他 ROS 节点协同工作，实现机器人控制和数据处理。

必须的知识：

- **ROS 框架：**熟悉 ROS 的基本概念、节点、话题、服务等。
- **C++ / Python 编程：**掌握 C++ 或 Python 编程，能够开发 ROS 节点并与控制器进行交互。
- **机器人控制：**了解 ROS 中常见的控制方法和工具（如 MoveIt、RViz 等）。

适合的用户：ROS 开发人员，尤其是已经熟悉 ROS 环境并希望将纳博特控制器集成到 ROS 系统中的工程师。

总结：

- **选择开发形式：**根据你的需求（如算法定制、UI 定制、控制器集成等），选择合适的二次开发形式。
 - **必须掌握的技术：**不同的开发方式需要掌握不同的技术栈，选择开发形式后，深入学习相应的编程语言和平台。
-

不同目标受众适合的开发方式

纳博特系统为不同类型的用户提供了灵活的开发平台，能够满足从初学者到专业研发者的需求。根据不同的目标受众，我们推荐以下的开发方式，以帮助你更快速地上手并实现目标。

1. 本体厂商

目标需求：快速客制化控制系统，打造自有机器人大脑，提高品牌辨识度。

推荐开发方式：

- **控制器二次开发：**
 - 本体厂商通常需要定制化的控制算法以适配其硬件，提升产品的独特性和市场竞争力。控制器二次开发允许对底层控制进行深度定制，提供强大的性能优化空间。
 - 可以通过控制器二次开发，调整机器人运动控制、插补算法等，以实现特定功能，确保机器人的精准性和高效性。
- **示教器二次开发：**
 - 若本体厂商需要打造一个专属于自己的用户界面，提升操作体验，示教器二次开发将是不错的选择。
 - 可根据品牌需求进行界面设计，定制图形界面，提高用户的使用体验，进而增强品牌辨识度。

技术要求：

- 对硬件有较深了解，熟悉 C++ 编程和控制算法。
-

2. 集成商

目标需求：开发专有工艺，保护自有行业知识，打造产品护城河。

推荐开发方式：

- **上位机开发：**
 - 集成商通常需要在多个设备之间进行高效通信和控制，且需要集成自己的专有工艺和技术。上位机开发能够帮助集成商在 PC 或上位机上进行复杂的控制，并与控制器进行深度集成。
 - 使用上位机开发可以通过 API 进行机器人控制，同时收集和处理来自机器人的数据，便于对自有工艺进行优化。
- **JSON 协议开发：**
 - JSON 协议是一种轻量级的通信方式，适合跨平台、跨系统的应用场景。集成商可以利用 JSON 协议进行标准化的数据传输，保护自有工艺数据，避免对外泄露。
 - 通过标准化协议，集成商可以实现与多个不同系统的兼容性，提升产品的可扩展性。

技术要求：

- 熟悉网络协议和数据交换，能够进行系统集成和高效数据传输。
-

3. 科研用户

目标需求：专注高精度运控算法研究，深挖机器人极限性能，勇攀科技高峰。

推荐开发方式：

- **控制器二次开发：**
 - 科研用户需要对控制算法进行深入研究和定制。控制器二次开发为其提供了一个强大的平台，能够根据科研需求对控制系统进行调整和优化。
 - 可以深度定制运动控制和算法，进行高精度的控制策略测试，探索机器人在极限性能下的表现。
- **ROS 开发：**
 - ROS（机器人操作系统）是科研领域的常用开发框架，适合于需要进行复杂算法和系统集成的用户。科研用户可以通过 ROS 系统对机器人的运动规划、传感器数据处理等进行高层次的研究和实验。
 - ROS 开发能够支持多机器人的协同工作，为科研用户提供丰富的工具库和开发支持，帮助他们在机器人领域实现创新。

技术要求：

- 高度的技术研发能力，熟悉控制算法、ROS 开发和高精度运动控制。
-

4. 教育用户

目标需求：培养学生不仅会用机器人，还能自主设计和制造机器人。

推荐开发方式：

- **示教器二次开发：**
 - 教育用户可以通过示教器二次开发，定制简单易用的操作界面，让学生更好地理解机器人控制和编程过程。
 - 示例程序和易于理解的图形界面将帮助学生快速上手，同时也能为他们提供一些基础的编程任务，增强他们的动手能力。
- **JSON 协议开发：**
 - 对于教学需要，JSON 协议的简单易用性可以帮助学生快速掌握数据传输和机器人控制的基本概念。通过简化的数据交换流程，学生可以在较短的时间内理解机器人与其他系统的互动方式。
 - 同时，JSON 协议也为学生提供了跨平台的开发体验，让他们能够更好地与不同的硬件平台进行对接。

技术要求：

- 对机器人操作有基本了解，熟悉图形界面开发和基础编程语言。

总结：根据不同的需求，选择合适的开发方式

目标受众	推荐开发方式	开发侧重点
本体厂商	控制器二次开发、示教器二次开发	深度定制控制系统和用户界面
集成商	上位机开发、JSON 协议开发	高效通信和自有工艺保护
科研用户	控制器二次开发、ROS 开发	精确控制和复杂算法研究
教育用户	示教器二次开发、JSON 协议开发	易用性和跨平台数据传输

入门小贴士：

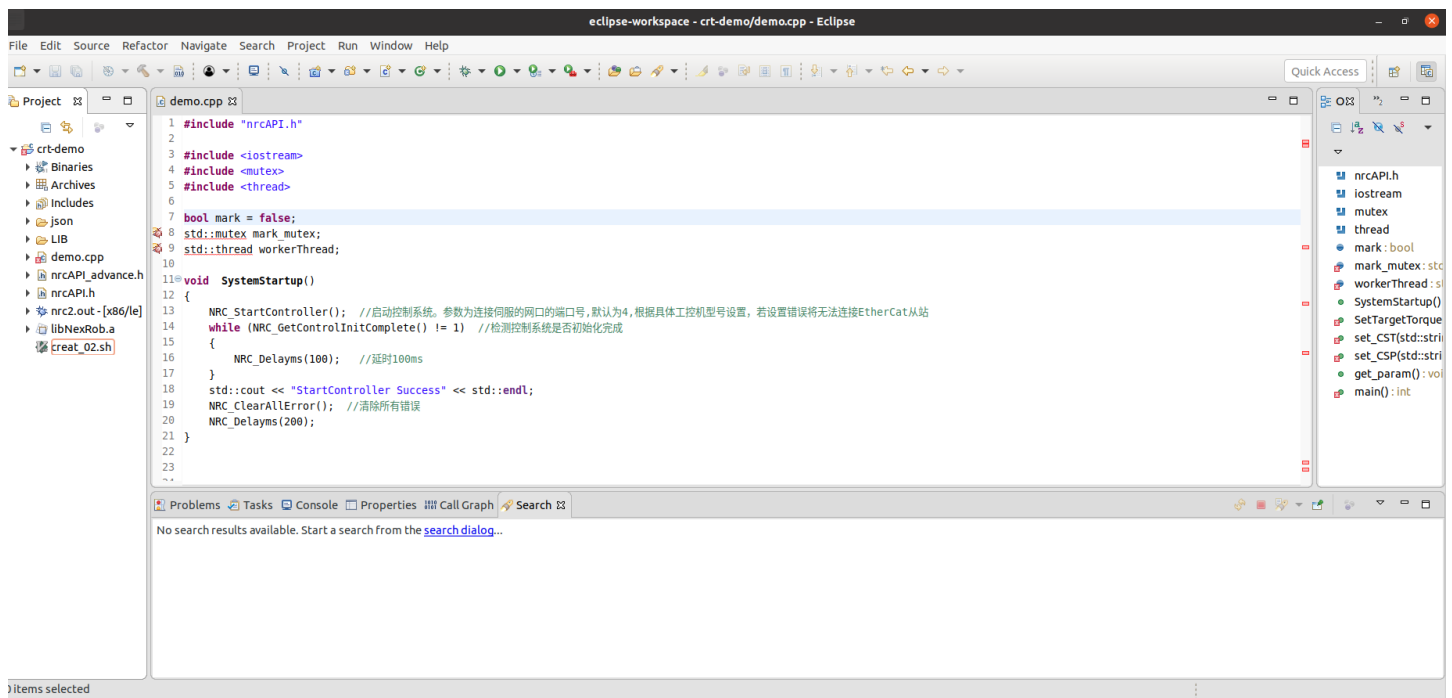
- **学习资料：**各开发方式都提供了丰富的文档和示例代码，建议在开始之前先通读相关文档，了解开发平台的基础知识。

- **社区支持**：对于科研和教育用户，ROS 和 JSON 协议有广泛的社区支持，可以通过在线论坛、教程和资源共享，帮助你更好地理解和使用平台。
- **灵活应用**：针对不同的目标受众，建议选择灵活且易于扩展的开发方式，尤其在初期阶段，保持开发路径的开放性有助于后期的快速迭代。

根据你的需求和技术背景，选择合适的开发路径，并根据项目需求逐步深入，能够使你在短时间内实现目标，并获得更好的开发体验。

Demo示例

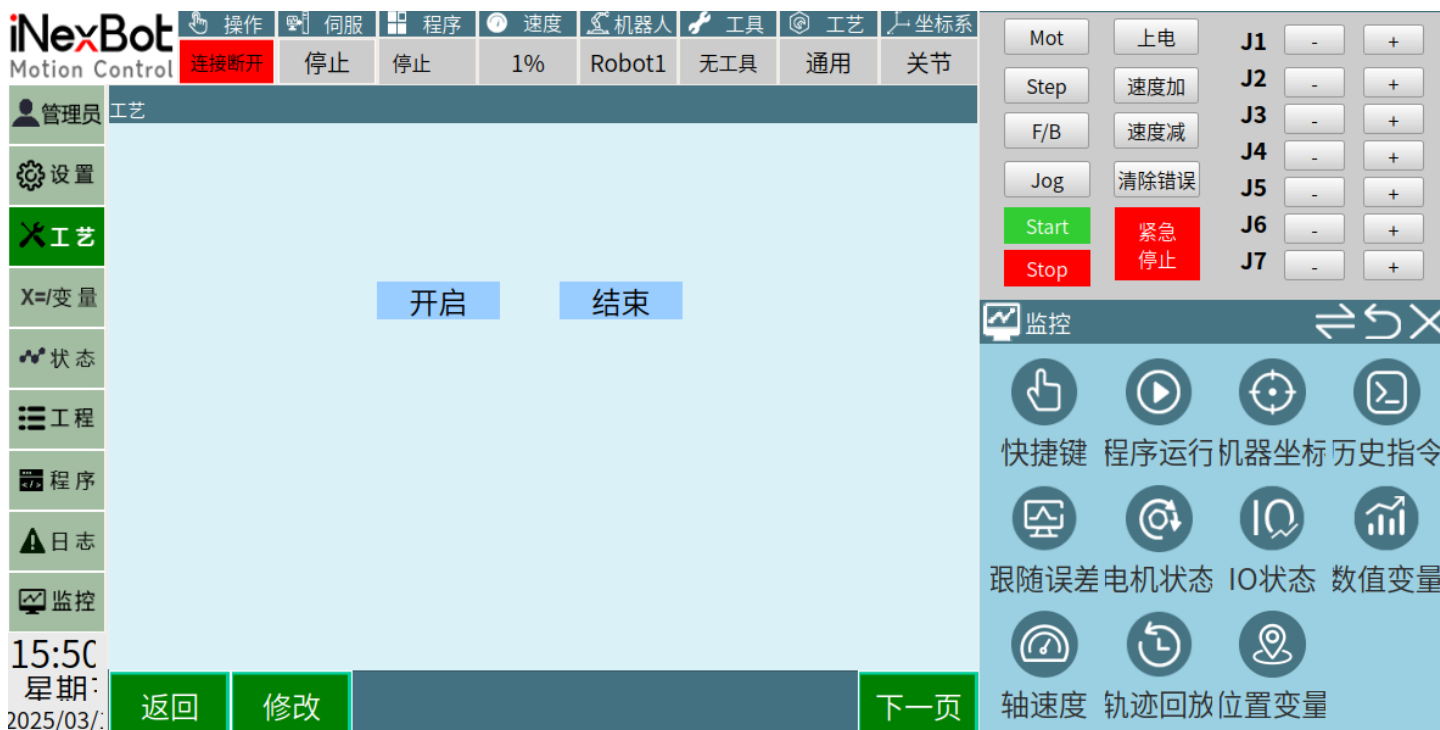
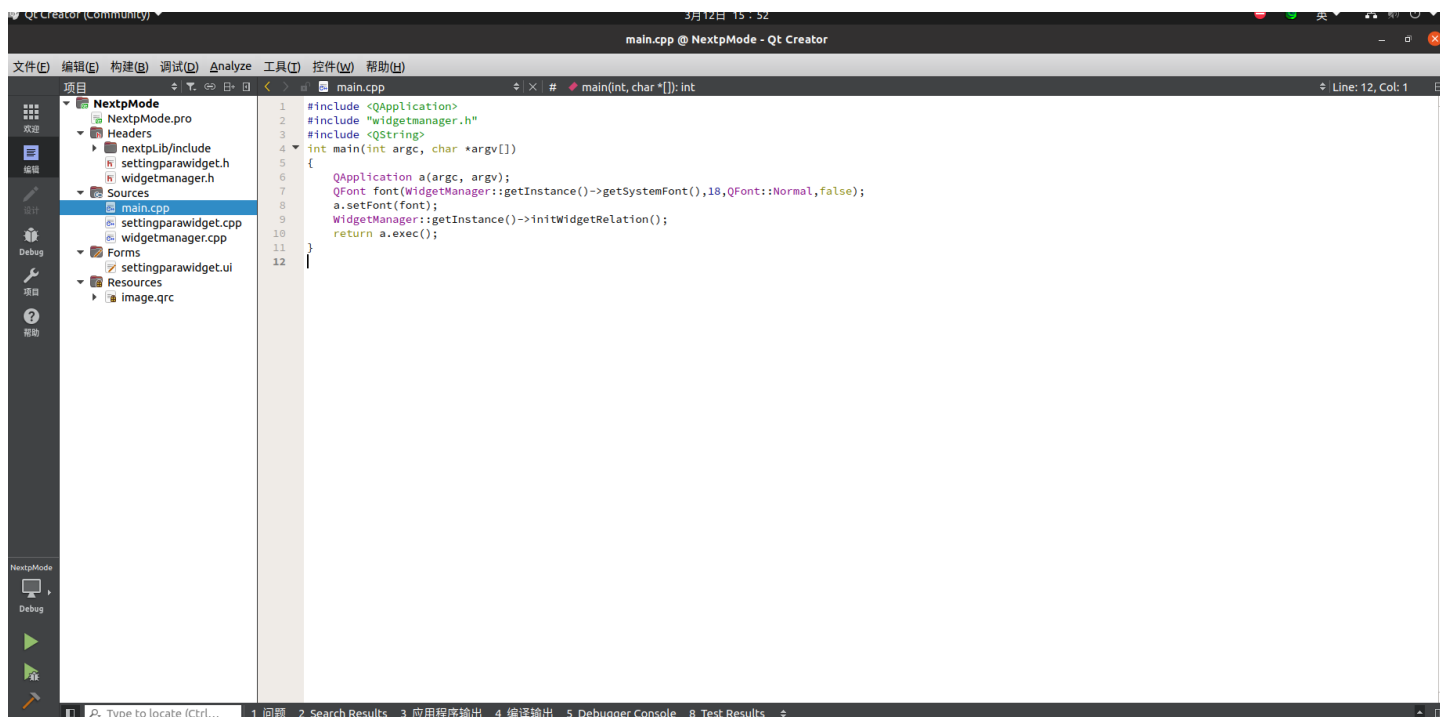
1. 控制器二次开发



下载

[控制器二次开发demo下载](#)

2. 示教器二次开发

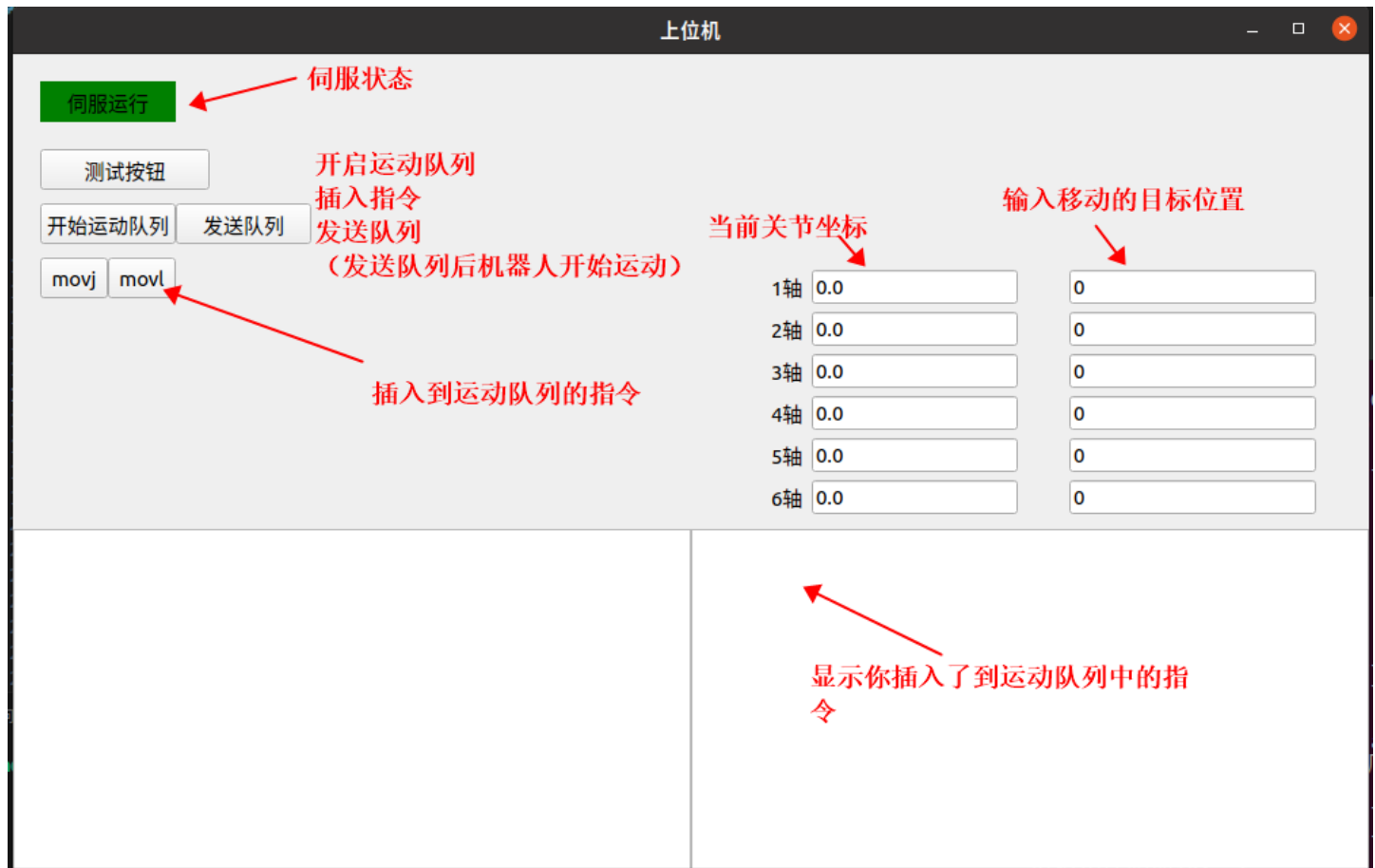


下载

[示教器二次开发demo下载](#)

3.上位机二次开发

- Python



下载

[python上位机二次开发demo下载](#)

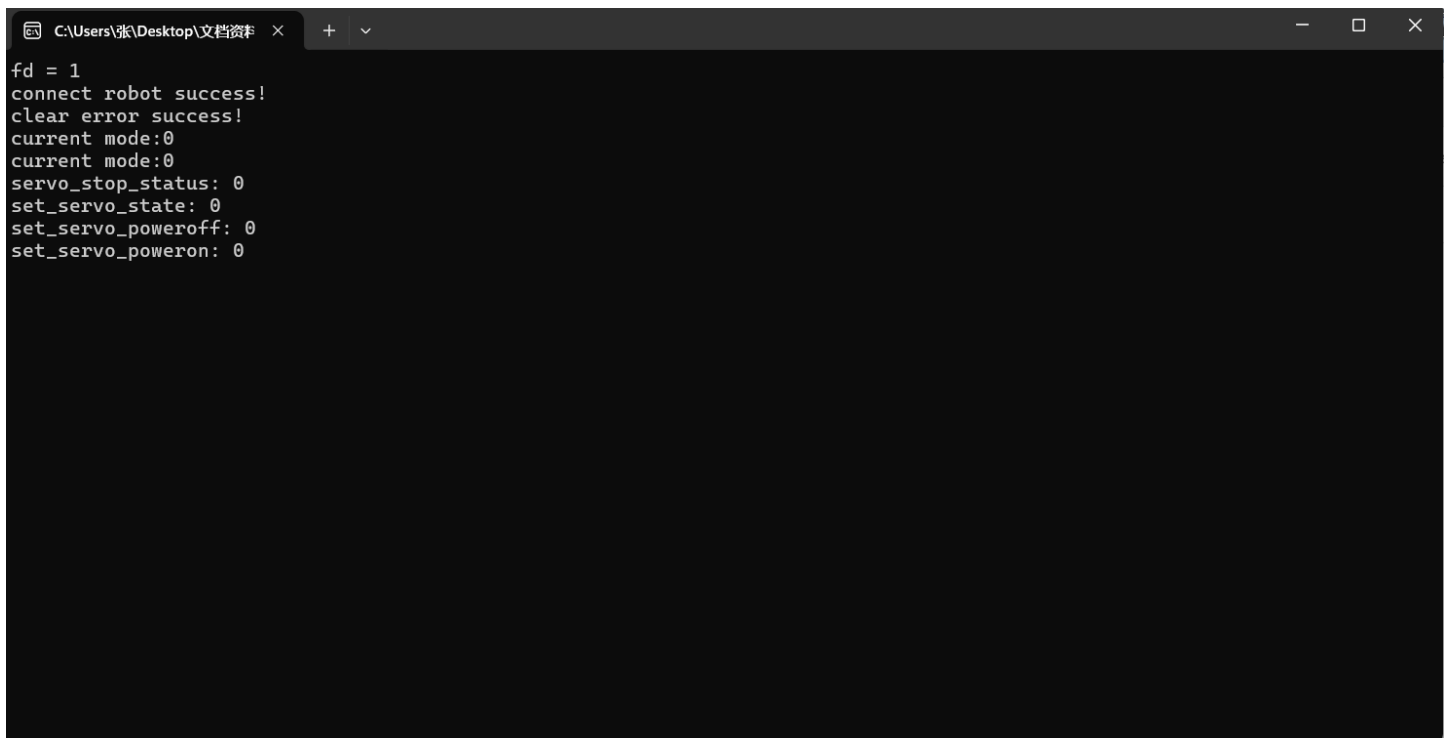
- C#



下载

[C#上位机二次开发demo下载](#)

- C++



A screenshot of a Windows command prompt window. The title bar shows the file path "C:\Users\张\Desktop\文档资料" and standard window controls. The command prompt displays the following output from a C++ program:

```
fd = 1
connect robot success!
clear error success!
current mode:0
current mode:0
servo_stop_status: 0
set_servo_state: 0
set_servo_poweroff: 0
set_servo_poweron: 0
```

下载

[C++上位机二次开发demo下载](#)

4.JSON协议二次开发



下载

[JSON协议二次开发demo下载](#)

5. HAL二次开发

```
=== 注册设备 ===
```

```
=== 设备查询 ===
```

```
所有设备：
```

```
Devices:
```

```
- AIO_1
```

```
数字IO设备：
```

```
Devices:
```

```
=== 设备使用示例 ===
```

```
请输入命令（按回车退出）：
```

```
测试 数字IO功能           请输入数字           1
```

```
测试 pwm设备功能          请输入数字           2
```

```
测试 编码器设备功能       请输入数字           3
```

```
测试 ups设备功能          请输入数字           4
```

```
测试 模拟can功能          请输入数字           5
```

```
输入回车键退出测试demo
```

下载

[HAL二次开发demo下载](#)